

La Programmation en langage C

Chapitre 3: les instructions de contrôle

La cible : MIP S3

Pr S.ELFILALI

Introduction

Les instructions de contrôle

servent à

- contrôler le déroulement de l'enchaînement des instructions à l'intérieur d'un programme

peuvent être

- des instructions conditionnelles
- des instructions itératives.

Les instructions conditionnelles

Les instructions conditionnelles

Les instructions conditionnelles permettent de :

- réaliser des tests
- et suivant le résultat de ces tests
 - d'exécuter des parties de code différentes.

C offre deux types d'instructions conditionnelles :

- l'instruction **if**
- l'instruction **switch**

Instruction conditionnelle if

l'instruction if-else

- **if** (*<expression>*) *<instruction1>*
- **if** (*<expression>*) *<instruction1>*
else *<instruction2>*

Instruction conditionnelle if

Description :

Instruction1 et
instruction2 :

- simple
- bloc
- instruction structurée

Si le résultat de
l'expression est « vrai »

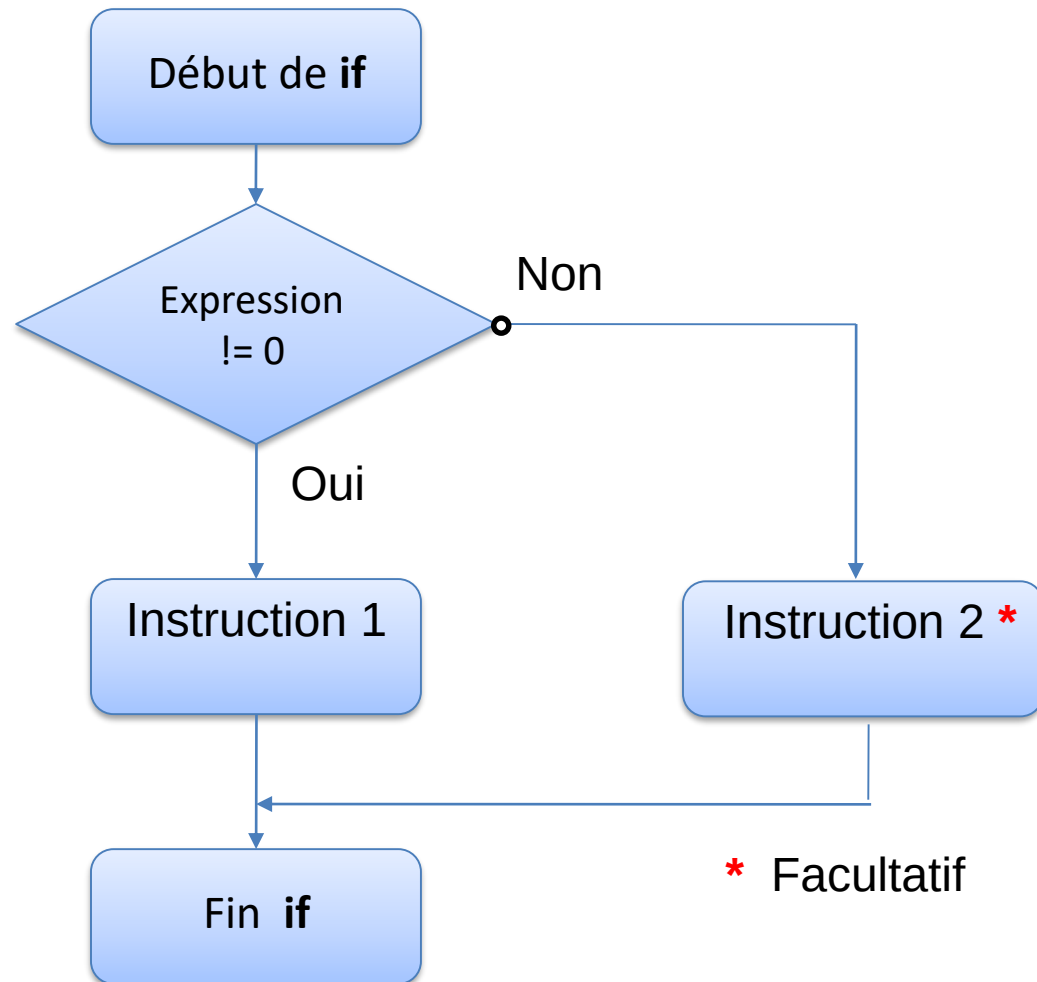
- on exécute l'instruction *<instruction1>*,

Dans le cas contraire

- on exécute l'instruction *<instruction2>* si elle existe.

Puis, le programme passe à l'exécution de l'instruction suivante.

Diagramme syntaxique du if :



À quoi sert l'instruction if ?

Exemple1 :

```
//declaration des variables
```

```
int qte_cmd ;
```

```
float prix, remise=0 ;
```

```
//lecture des donnees
```

```
printf("La quantité commandée : ") ; scanf("%d",&qte_cmd) ;
```

```
printf("Prix unitaire : ") ; scanf("%f",&prix) ;
```

```
//traitement
```

```
if(qte_cmd > 100) remise = 0.1;
```

```
//affichage des resultats
```

```
printf("Prix à payer : %.2f",prix*qte_cmd*(1 - remise) ) ;
```

Exemple2 :

```
//declaration des variables
```

```
char car ;
```

```
//lecture des donnees
```

```
printf("Tapez une lettre minuscule non accentuée : ");  
scanf(" %c",&car);
```

```
//traitement et affichage des resultats
```

```
if (car == 'a' || car == 'e' || car == 'i' || car == 'o' || car == 'u' || car == 'y')  
printf("la lettre %c est une voyelle",car);  
else  
printf("la lettre %c est une consonne",car);
```


Exemple3 :

```
//declaration des variables
```

```
.....
```

```
//lecture des donnees
```

```
.....
```

```
.....
```

```
//traitement et affichage des resultats
```

```
if (qte_cmd <= qte_stock)
{
    printf("Prix total = %.2f", qte_cmd*prix);
    qte_stock -= qte_cmd;
}
else
    printf("\aLa quantité commandée est sup. à la quantité en stock ! ");
```

Exercice 1 :

L'utilisateur saisit un caractère, le programme teste s'il s'agit d'une lettre majuscule, si oui il affiche cette lettre en minuscule, sinon il affiche un message d'erreur.

..... (tolower, toupper)

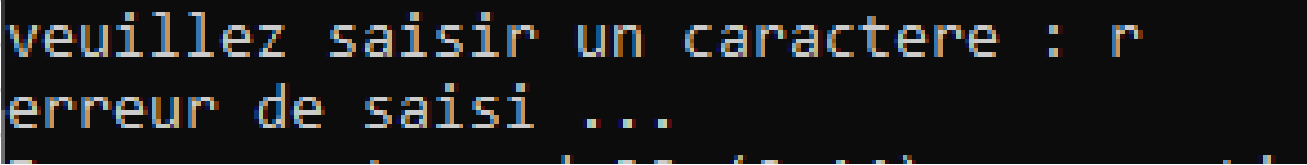
Solution :

```
#include <stdio.h>
void main(void)
{
    char ch;

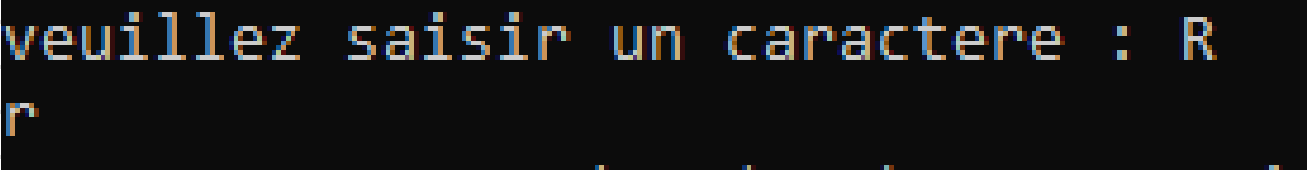
    printf("veuillez saisir un caractere : "); scanf("%c",&ch);

    if(ch==toupper(ch)) printf("%c",tolower(ch).);
    else printf("erreur de saisi ... ");

}
```



```
veuillez saisir un caractere : r
erreur de saisi ...
```



```
veuillez saisir un caractere : R
r
```

Exercice 2 :

Quel résultat affiche le programme suivant :

```
#include<stdio.h>
void main()
{
    int x = 3 ;

    if (x > 0)
    {
        x = -x ;
        printf("x = %d ", x);
    }
    if (x == 4) printf("x = %d", x);
}
```

Solution :

.....

.....**x=-3**

Quel résultat affiche le programme suivant :

```
#include<stdio.h>
void main()
{
    int x = 4 ;

    if (x > 0)
    {
        x = -x ;
        printf("x = %d ", x);
    }
    if (x == 4) printf("x = %d", x);
}
```

Solution :

.....**x=-4**

Imbrication d'instructions if

Il est possible d'imbriquer une instruction **if** à l'intérieur d'une autre.

Syntaxe :

```
if (<expression1>) if (<expression2>) <instruction1> else <instruction2>
```

Exemple :

```
if (a < b) if (c < b) z = b ; else z = a ;
```

Question : A quel « **if** » se rapporte « **else** » ?

Règle : le **else** se rapporte au dernier **if** rencontré auquel

un **else** n'a pas encore été attribué sauf si on utilise les accolades.

Exemple

La bonne mise en page du code ci-dessus est :

```
if (a < b)
  if (c < b)
    z = b ;
  else
    z = a ;
```

On peut mettre des { } pour rendre les choses plus claires.

```
if (a < b)
{
  if (c < b)
    z = b ;
  else
    z = a ;
}
```

Si l'on veut que « else » se rapporte au premier « if » :

```
if (a < b)
{
  if (c < b)
    z = b ;
}
else
  z = a ;
```

L'instruction if-else if

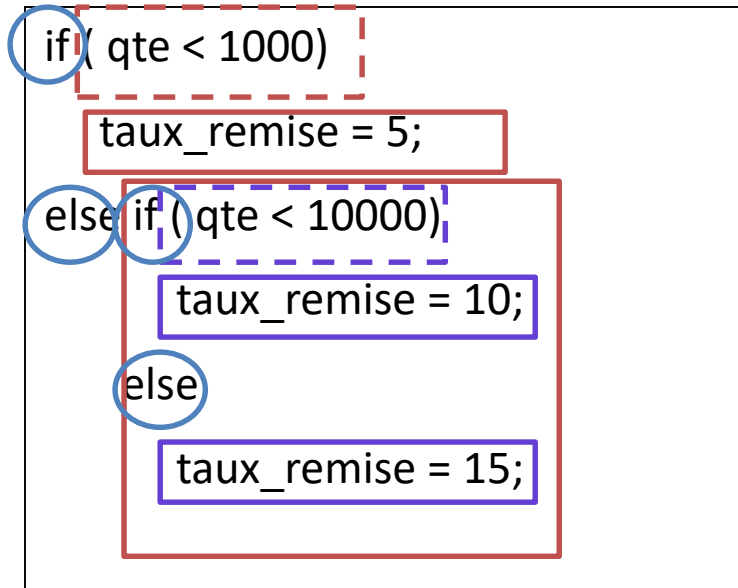
Il est fréquemment utile d'utiliser une série d'instructions **if-else if** pour répondre à une situation dans laquelle les choix sont multiples.

Syntaxe :

```
if (<expression1>) <instruction1>  
else if (<expression2>) <instruction2>  
else if (<expression3>) <instruction3>  
...  
[else    <instruction_n+1>]
```

NB : Chaque instruction est associée à une condition et le dernier else, s'il existe, correspond au cas où aucune condition n'est vraie.

Exemples :



Exemples :

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    char c;
```

```
    printf("Entrez une lettre non accentuée : ");
```

```
    scanf(" %c", &c);
```

```
    if (c >= 'A' && c <= 'Z') {
```

```
        printf("Cette lettre en minuscule est : %c", c+32);
```

```
    } else if (c >= 'a' && c <= 'z') {
```

```
        printf("Cette lettre en majuscule est : %c", c-32);
```

```
    } else
```

```
        printf("Erreur ! Ce n'est pas une lettre non accentuée");
```

```
}
```

Exercice 3 :

Ecrivez un programme qui permet de saisir une valeur de type entière et indiquez à l'utilisateur si celle-ci est positive, négative ou nulle.

Solution :

This image shows a full page of a document template designed for writing. It features approximately 28 evenly spaced, thin grey horizontal lines across the entire width of the page. The background is a solid light blue-grey color, providing a clean and professional look suitable for various types of documents or reports.

L'opérateur ?:

Le langage C offre la possibilité de remplacer l'instruction conditionnelle **if-else** dans le cas d'une affectation conditionnelle de variable par l'opérateur conditionnel « ?: » et qui a l'avantage de pouvoir être intégré dans une expression.

Syntaxe :

expr1 ? expr2 : expr3

On évalue **expr1**:

Si **expr1** est vraie

- Résultat est la valeur de **<expr2>**

Si **expr1** n'est pas
vraie,

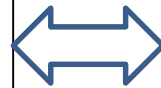
- Résultat est la valeur de **<expr3>**

Exemples :

```
int x = 5, y = 4, max;  
if (x>y)  
    max=x;  
else  
    max=y;  
printf("Le max est : %d",max) ;
```



```
int x = 5, y = 4, max;  
max = (x>y) ? x : y;  
printf("Le max est : %d",max) ;
```



```
int x = 5, y = 4;  
printf("Le max est : %d",(x>y) ? x : y) ;
```

Instruction conditionnelle switch

À quoi sert l'instruction switch?

L'instruction conditionnelle **switch** permet de remplacer plusieurs if-else imbriqués lorsqu'il s'agit d'effectuer un choix multiple.

Exemples :

switch (expression)

```
{  
  case expression_const_1 :  
    instruction_1;  
    [break;]  
  case expression_const_2 :  
    instruction_2;  
    [break;]  
    ....  
  
  case expression_const_n :  
    instruction_n;  
    [break;]  
  [default:  
    instruction_par_défaut;]  
}
```

NB :

expression et
expression_const_i

- ni de type réel
- ni de type chaîne de caractère.

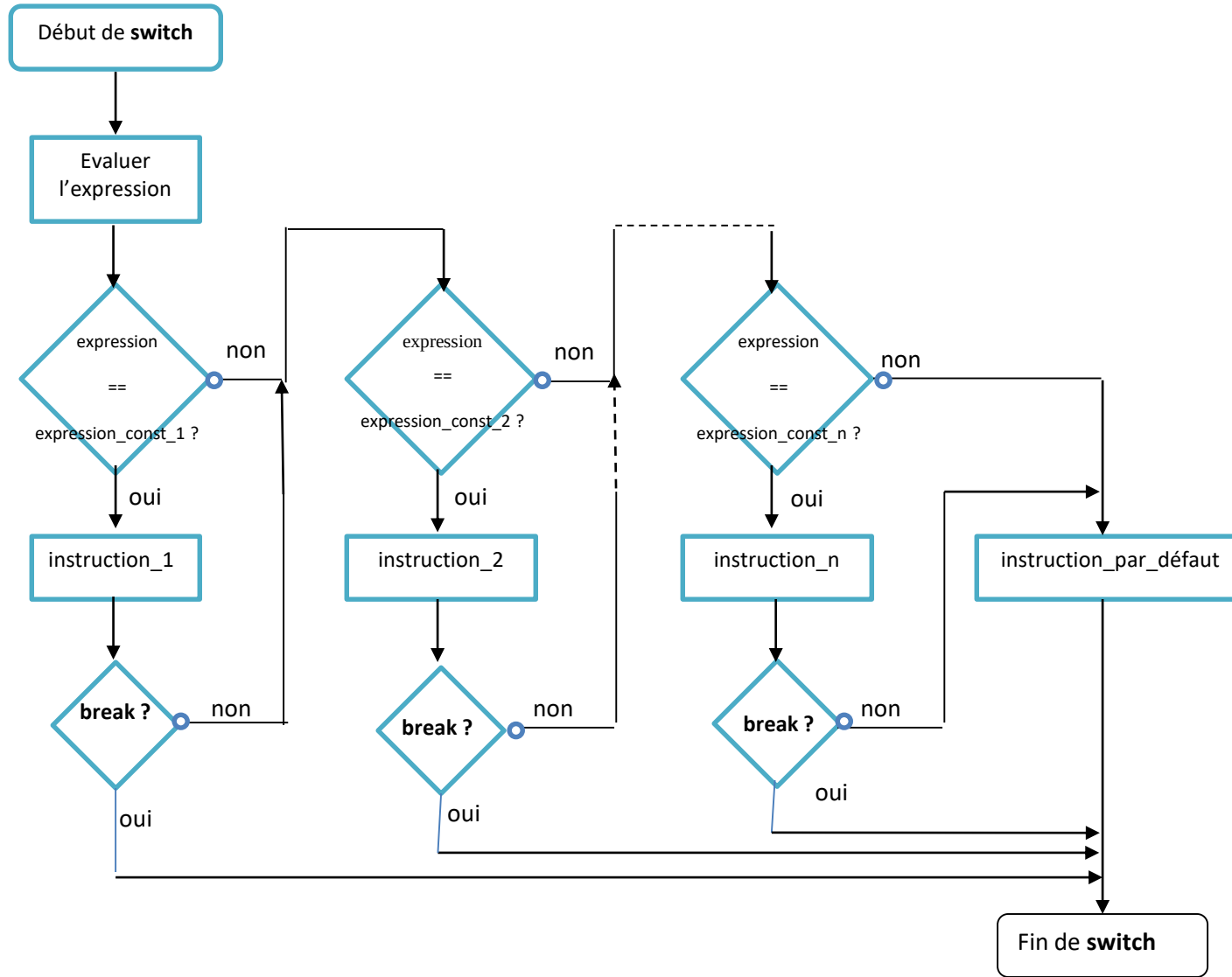
expression_const_i

- doivent obligatoirement être distinctes.

break et default

- sont optionnelles.

Diagramme syntaxique de switch :



Exemples :

```
void main()
{
    int jour;
    printf("Entrez le numéro d'un jour de la semaine (1 à 7) : "); scanf("%d",&jour);
    printf("Le jour %d de la semaine est le ",jour);
    switch (jour)
    {
        case 1 :      printf("DIMANCHE");
                       break;

        case 2 :      printf("LUNDI");
                       break;

        .....

        case 7 :      printf("SAMEDI");
                       break;

        default: printf("Erreur!");
    }
}
```

Exemples :

```
void main()
{
    int jour;
    printf("Entrez le numéro d'un jour de la semaine (1 à 7) : "); scanf("%d",&jour);
    printf("Le jour %d de la semaine est le ",jour);
        if (jour == 1)
            printf("DIMANCHE");
        else if (jour == 2)
            printf("LUNDI");
            else if (jour == 3)
                printf("MARDI");
.....
            else if (jour == 7)
                printf("SAMEDI");
            else
                printf("Erreur!");
}
```

Instructions itératives

Les instructions itératives ou boucles sont réalisées à l'aide d'une des trois instructions de contrôle suivantes :

Instructions itératives

do-while

while

for

Instruction do-while <faire-tant que> :

La boucle *do-while* permet de répéter une instruction ou un bloc d'instructions, un certain nombre de fois, tant qu'une condition est vérifiée.

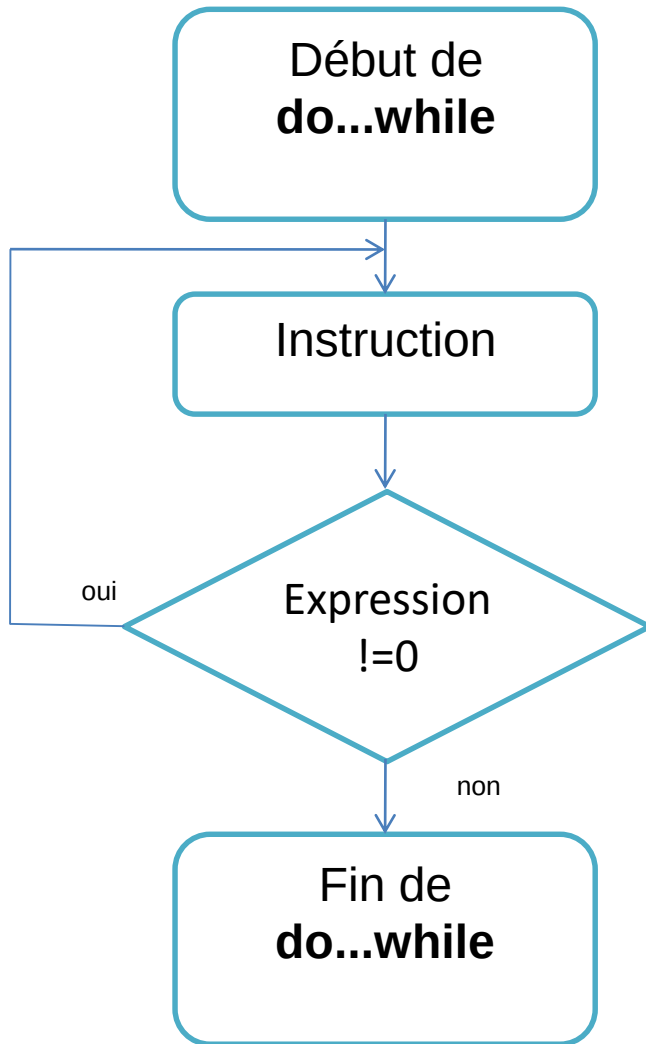
Syntaxe

do

<instruction> /*ou <bloc d'instructions> */

while (<expression>);

Diagramme syntaxique do-while :



Remarques :

L'<instruction> est exécutée au moins une fois,

- car le test de l'<expression> s'effectue à la fin de la boucle.

Il faut s'assurer que l'<instruction> modifie la valeur de l'<expression>

- si on veut éviter d'être pris dans une boucle sans fin.

Exemples :

```
void main ()
{
int compteur = 1;

do
{
    printf("La valeur du compteur vaut : %d\n", compteur);
        compteur++ ;
}while (compteur <=10);

printf("La valeur finale du compteur vaut : %d",compteur);
}
```

Exemples :

```
#define PI 22/7.0
void main ()
{
    float rayon ;
    do
    {
        printf("Donnez le rayon : ") ;scanf("%f",&rayon);
        if(rayon < 0) printf("\aLe rayon doit être positif\n") ;
    }while (rayon < 0);
    printf("Aire = %.2f",rayon*rayon*PI);
}
```


Exemples :

```
#include <stdio.h>
void main ()
{
    char rep ;
    do
    {
        ....
        printf("Vous-voulez continuer (o/n) : " ) ;
        scanf(" %c",&rep); /* rep = getche(); */

    }while (rep == 'o' || rep == 'O');
}
```

L'instruction while <tant que>

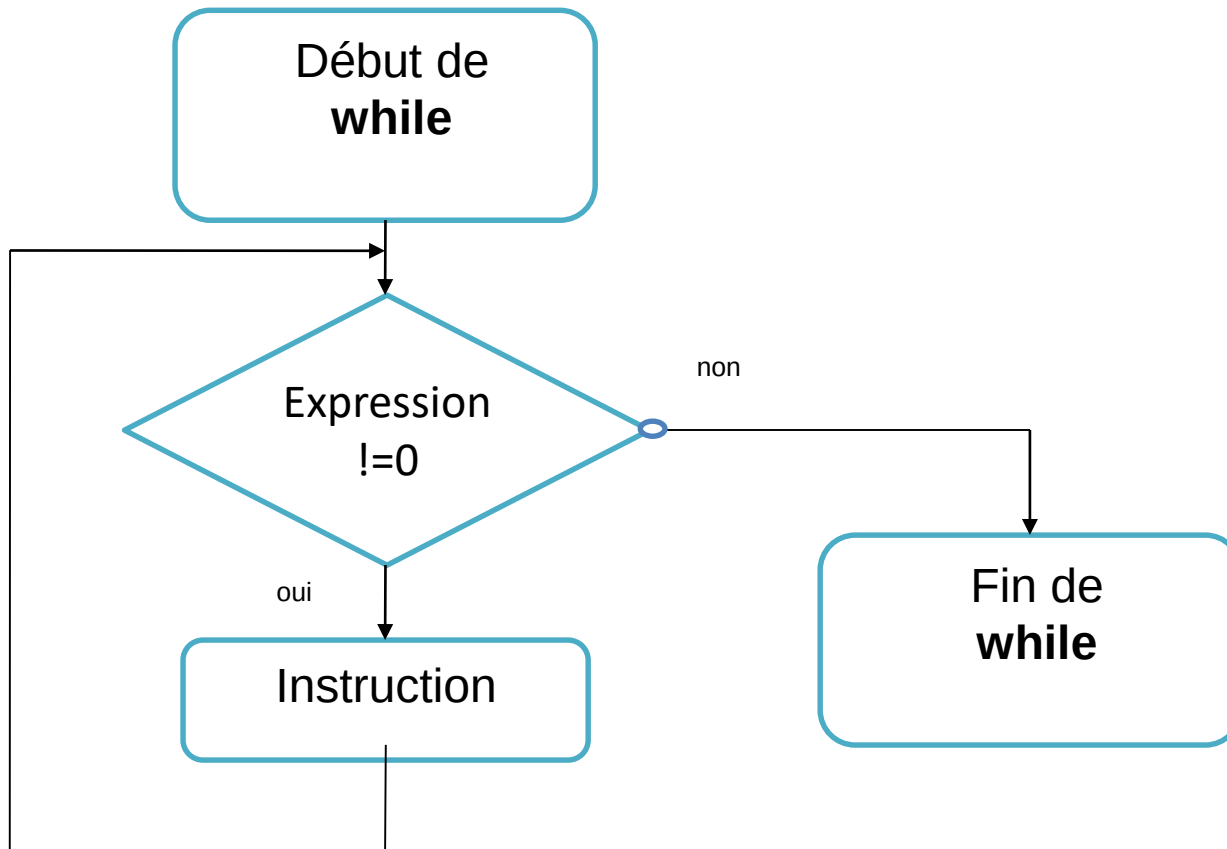
L'instruction **while** se comporte comme la boucle **do-while**, à la différence près que l'<expression> est évaluée en début de boucle. Il y a donc possibilité que l'<instruction> à répéter ne soit jamais exécutée, si le premier test n'est pas vérifié.

Syntaxe

While (<expression>)

<instruction> ; /*ou <bloc d'instructions> */

Diagramme syntaxique de while :



Exemples :

```
#include <conio.h>
void main ()
{
    int color = 1;
    while (color <= 15)
    {
        textcolor(color);
        cprintf("Couleur N° %d\n\r", color++);
    }
}
```

```
HANDLE hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);  
CONSOLE_SCREEN_BUFFER_INFO csbi;
```

```
    //We use csbi for the wAttributes word.
```

```
if(GetConsoleScreenBufferInfo(hStdOut, &csbi))
```

```
{
```

```
    //Mask out all but the background attribute, and add in the foreground
```

```
color
```

```
    wColor = (csbi.wAttributes & 0xF0) + (ForgC & 0x0F);
```

```
    SetConsoleTextAttribute(hStdOut, wColor);
```

```
}
```

```
}
```

```
void main ()
```

```
{
```

```
    int color =1;
```

```
    while(color<=15)
```

```
    {
```

```
        SetColor(color);
```

```
        printf("couleur N° %d \n\r", color++);
```

```
    }
```

```
}
```

Exercices :

Exercice 4 :

Ecrire un programme qui demande un entier positif n et qui affiche la somme des n premiers entiers.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int s=0, n,i;
    printf("donner un entier
positif : "); scanf("%d",&n);
    i=0;
    do
    {
        s+=i;
        i++;
    } while (i <= n);

    printf("la somme est :%d
",s);

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int s=0, n,i;
    printf("donner un entier
positif : ");
    scanf("%d",&n);
    i=0;

    while (i <= n)
    {
        s+=i;
        i++;
    }
    printf("la somme est :%d
",s);

    return 0;
}
```

```
#include <stdlib.h>
#include <stdlib.h>

int main()
{
    int s=0, n,i;
    printf("donner un entier
positif : ");
    scanf("%d",&n);

    for(i=0;i<=n;i++)
        s+=i;

    printf("la somme est :%d
",s);

    return 0;
}
```

Exercices :

Exercice 5

Ecrire un programme qui calcule la somme des nombres positifs donnés par l'utilisateur, le programme lira des nombres tant qu'ils seront positifs.

Solution :

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int s=0, n;
    do
    {
        printf("donner un entier positif : "); scanf("%d",&n);

        if(n>=0)    s+=n;

    } while (n >= 0);
    printf("la somme est :%d ",s);

    return 0;
}
```

Instruction itérative for <Pour> :

Syntaxe

```
for ([expression_1];[expression_2];[expression_3])  
<instruction>; /*ou <bloc d'instructions>*/
```

Description :

expression_1 : est évaluée une seule fois. Elle sert à initialiser les données de la boucle.

expression_2 : est la condition d'exécution de l'<instruction> à répéter. Elle est évaluée et testée avant chaque parcours de la boucle. Si son résultat est VRAI alors l'<instruction> est exécutée sinon la boucle est terminée

expression_3 : est évaluée après l'exécution de l'<instruction> à répéter. Elle est utilisée pour mettre à jour les données de la boucle.

Instruction itérative for <Pour> :

```
for ([expression_1];[expression_2];[expression_3])  
    <instruction>; /*ou <bloc d'instructions>*/
```

Description :

➤ La boucle **for** s'utilise avec **trois** expressions, **séparées par des points virgules** :

expression_1 :

- est évaluée une seule fois, au début de l'exécution de la boucle .
- Elle sert à initialiser les données de la boucle.

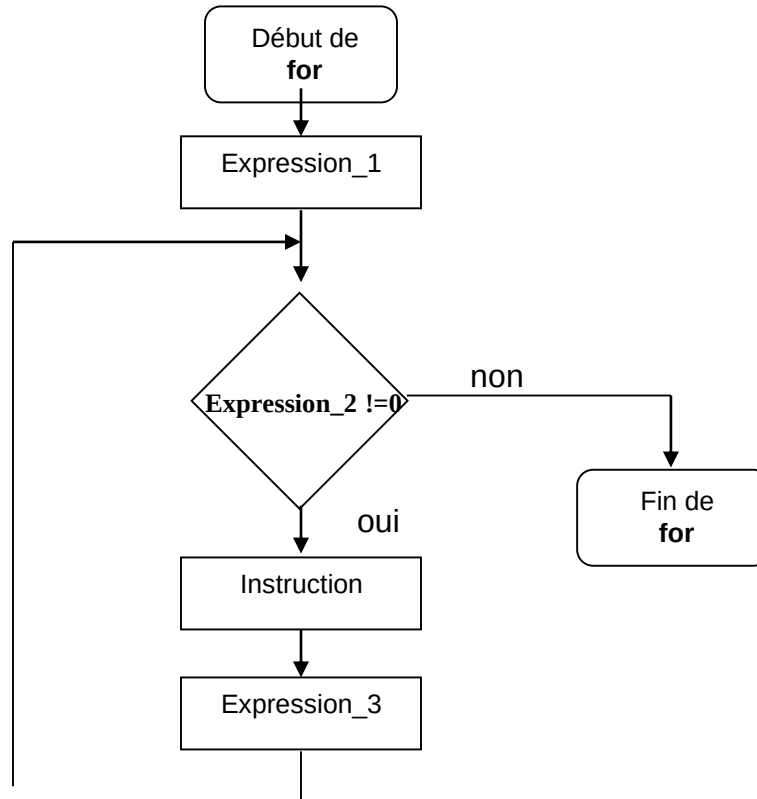
expression_2 :

- est la condition d'exécution de l'<instruction> à répéter.
- Elle est évaluée et testée avant chaque parcours de la boucle.
- Si son résultat est VRAI alors l'<instruction> est exécutée
- sinon la boucle est terminée.

expression_3 :

- est évaluée après l'exécution de l'<instruction> à répéter.
- Elle utilisée pour mettre à jour les données de la boucle.

Diagramme syntaxique



Instruction itérative for <Pour> :

Remarques :

- La boucle for est équivalente à la structure suivante :

```
expression_1;  
    while (expression_2)  
    {  
        instruction ;  
        expression_3;  
    }
```

- Par rapport à la boucle while, la boucle for met en valeur l'<expression_1> et l'<expression_3>.
- Expression_1, expression_2 et expression_3 sont facultatives.
- Si l'<expression_2> est omise alors la boucle est infinie.
- Quand tout le traitement peut être spécifié dans les trois expressions du for, l'instruction à répéter peut se réduire à une instruction vide.

Exemples :

```
#include <stdio.h>
void main ()
{
    int m,i ;
    printf("Donnez le multiplicande compris entre 1 et 9 : ") ;
    scanf("%d",&m);
    printf("\n\t\t\tTable de multiplication par %d\n",m) ;
    for (i=1; i <= 10 ; i++)
        printf("%2d x %2d = %2d\n",m,i,m*i) ;
}
```

Exercices :

Exercice 6 :

Ecrire un programme qui affiche la somme des **n** premiers termes d'une progression arithmétique de raison **r** et de premier terme **m**.

Par exemple, si **n** = 7, **r** = 3 et **m** = 23 alors

$$\text{somme} = 23 + 26 + 29 + 32 + 35 + 38 + 41 = \mathbf{224}$$

$$= (m+0*r)+(m+1*r)+(m+2*r) + (m+3*r) + \dots (m+n*r)$$

Solution :

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>
void main()
{
    int i,n,m,r,s;

    printf("donner le premier terme de la suite : ");scanf("%d",&m);
    printf("donner la raison de la suite : ");scanf("%d",&r);
    printf("donner le nombre de n elements de la suite : ");scanf("%d",&n);

    s=0;
    for (i=0;i<n;i++)

        s=s+(m+i*r);

    printf("la somme de %d premier terme de la suite de la raison %d et du premier terme  %d est : %d ", n,r,m,s);
    getch();
}
```


Remarque sur les instructions itératives

Les différentes boucles peuvent être imbriquées.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int m,n ;

    for (m=1; m <= 9 ; m++)
    {
        clrscr();
        printf("\t\t\tTable de multiplication par %d\n\n",m) ;
        for (n=1; n <= 10 ; n++)
            printf("\t\t\t\t\t%2d x %2d = %2d\n",m,n,m*n);
        getch();
    }
}
```

Lorsque l'on imbrique des instructions for, il faut veiller à ne pas utiliser le même compteur pour chacune des instructions.

Les instructions de rupture de séquences

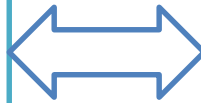
Ces instructions permettent de rompre le déroulement séquentiel d'une suite d'instructions.

L'instruction break

L'instruction **break** provoque le passage à l'instruction qui suit immédiatement le corps de la boucle *while*, *do-while*, *for* ou *switch*.

```
#include <stdio.h>

void main()
{
    int i;
    for(i=0; ;i++)
    {
        printf("La valeur du compteur vaut :
%d\n", i);
        if(i == 10) break;
    }
}
```



```
#include <stdio.h>

void main()
{
    int i = -1;
    do
    {
        printf("La valeur du compteur vaut :
%d\n", ++i);
        if(i == 10) break;
    } while(1) ;
}
```

L'instruction break

Que se passe -t-il avec le code suivant?

```
#include <stdio.h>
void main()
{
    int i;
    for(i=0;i<=20 ;i++)
    {
        printf("La valeur du compteur vaut :
%d\n", i);
        if(i == 10) break;
    }
}
```

L'instruction break

Remarques :

- L'instruction break ne peut être utilisée que dans le corps d'une boucle ou d'un switch.
- L'action de break ne s'applique qu'à la boucle la plus intérieure dans laquelle elle se trouve.
- Elle ne permet pas de sortir de plusieurs boucles imbriquées.

L'instruction goto

Cette instruction permet de se brancher (inconditionnellement) à une étiquette (identificateur) à l'intérieur de la même fonction.

Sa syntaxe est la suivante :

```
goto étiquette ;
```

Et la déclaration d'une étiquette se fait de la manière suivante

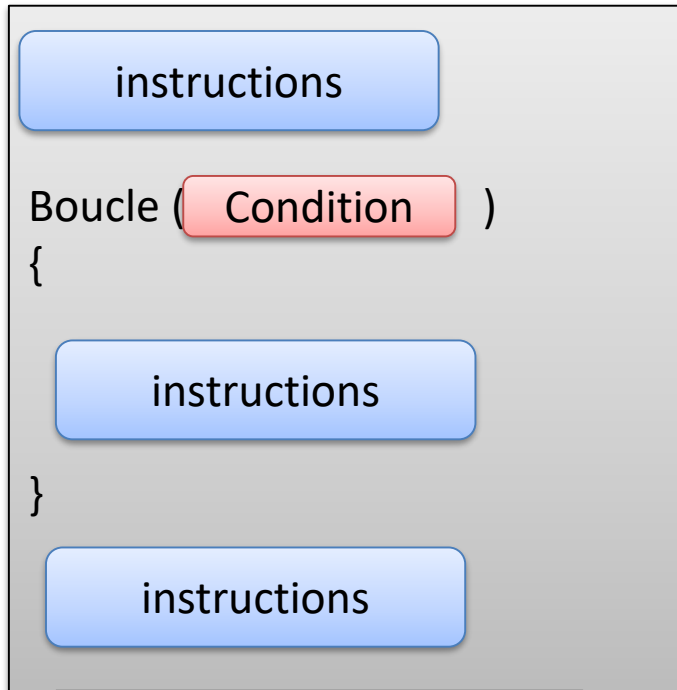
```
étiquette :  
    instruction ;
```

Examples :

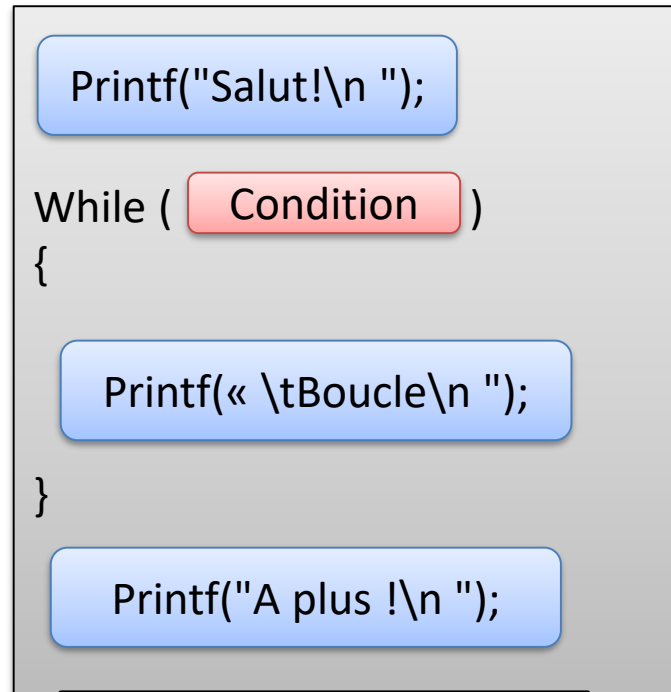
```
void main()
{
    int i,j,k;
    for(i=1 ;i<=5 ;i++)
        for(j=1;j<=5;j++)
            for(k=1;k<=5;k++)
            {
                printf("i = %d\t j = %d\t k = %d \n",i,j,k);

                if (i*j*k== 40) goto sortie;
            }
sortie : printf("Fin du programme") ;
}
```

Résumons : Les boucles



Structure



Exemple

Salut !

Boucle
Boucle
Boucle
Boucle
Boucle

A plus !

Boucles en C : while, do while et for

Résumons : Les boucles

C'est quoi une boucle ?

Boucle c'est un ensemble d'instructions qui est exécuté un certain nombre de fois tant que la condition liée est vraie

Résumons : Les boucles

La boucle while

```
While ( Condition )  
{  
    instructions  
}
```

Structure

```
Int var =0;  
While ( var < 5 )  
{  
    Printf("%d\n" , var);  
    Var++;  
}
```

Exemple

0
1
2
3
4

Résumons : Les boucles

La boucle while

Exemple : compte à rebours

Passer sur codeblocks et faire un programme sans boucle et avec la boucle

Utiliser les fonctions sleep et générer le bip par `printf("\a");`

la fonction `Sleep("temps en millisecondes");`, permet de faire attendre le programme pendant un certains nombre de millisecondes avant de continuer à exécuter les instructions.

La boucle while

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int compte=10;
    printf("\a"); // bip
    getch();
    printf("explosion dans 10 s");

    printf("\ncompte à rebours ..... !\n");

    printf("\n%d",compte);
    compte--;
    sleep(1); // attente en millisecondes

    printf("\n%d",compte);
    compte--;
    sleep(1);

    printf("\n%d",compte);
    compte--;
    sleep(1);

    .....

    printf("\n%d",compte);
    compte--;
    sleep(1);

    printf("\n%d",compte);
    compte--;
    sleep(1);
    printf("\a");

    printf("\nBOOOOOOMMM");
    return 0;
}
```

La boucle while

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int compte=10;
    printf("\a");
    getch();
    printf("explosion dans 10 s");

    printf("\ncompte à rebours ..... !\n");

    while (compte>0)
    {
        printf("\n%d",compte);
        compte--;
        sleep(1);
    }

    printf("\a");

    printf("\nBOOOOOOOMMM");
    return 0;
}
```

Résumons : Les boucles

La boucle do while

```
do  
{  
    instructions  
} While ( Condition );
```

Structure

```
Int var =6;  
do  
{  
    Printf("%d\n" , var);  
    Var++;  
} While ( var < 5 );
```

Exemple

6

Résumons : Les boucles

La boucle do while

Exemple : contrôle de saisie

Passer sur codeblocks et faire un programme qui demande à l'utilisateur de saisir un nombre entre 1 et 10

Résumons : Les boucles

La boucle do while

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int saisie_utilisateur = 0;
    do
    {
        printf("donner un nombre entre 1 et 10 : ");
        scanf("%d",&saisie_utilisateur);
    }
    while (saisie_utilisateur <1 || saisie_utilisateur>10);
    return 0;
}
```


Résumons : Les boucles

La boucle for

```
for (int var =0 ; var<5 ; var++ )  
{  
    printf("%d\n",var);  
}
```

0
1
2
3
4

Exemple

```
for ( init ; Condition ; modification )  
{  
    instructions  
}
```

Structure

```
int var =0 ;  
while(var<5)  
{  
    printf("%d\n",var);  
    var ++ ;  
}
```

0
1
2
3
4

Equivalent

Résumons : Les boucles

La boucle for

Exemple : compte à rebours

Passer sur codeblocks et faire un programme avec la boucle for

Utiliser les fonctions sleep et générer le bip par `printf("\a");`

la fonction `Sleep("temps en millisecondes");`, permet de faire attendre le programme pendant un certains nombre de millisecondes avant de continuer à exécuter les instructions.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
int compteur = 10;
    printf("Explosion dans  ");
    for( compteur = 10 ; compteur>0 ; compteur--)
    {
        printf("\n%d", compteur);
    }
    printf("\n\na BOOM");
    return 0;

}
```

Règles d'or des boucles

Comment choisir la boucle qu'on va utiliser ?

Si on connaît le nombre d'itération du bloc d'instruction alors on utilise la boucle for

Si on connaît pas le nombre d'itération du bloc alors nous avons deux choix :

Si on veut exécuter au moins une fois le bloc d'instruction alors c'est la boucle « do .. While »

Et dans tous les autres cas , on utilise la boucle while

Règles d'or des boucles

Boucle infinie

C'est une boucle qui va s'exécuter à l'infini!

Cad que sa condition ne va jamais passer à faux!!
Le programme va se bloquer !!

Le seul moyen pour le débloquent : **Ctrl+C** ou **Alt F4**

Exemple

Apportez les modification suivantes au programme et commentez :

1. Compteur ++
2. Modifier le type en short et compteur ++
3. Modifier la condition : >-99999 et compteur --

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int compteur = 10;
    printf("Explosion dans ");
    for( compteur = 10 ; compteur>0 ; compteur --)
    {
        printf("\n%d", compteur);
    }
    printf("\n\na BOOM");
    return 0;
}
```

Règles d'or des boucles

Boucle infinie

Apportez les modifications suivantes au programme et commentez :

1. **Compteur ++** : boucle infinie, on n'arrive pas à voir le max de ce type, on doit l'arrêter par Ctrl+C ... la condition peut être fautive au max de ce type
2. **Modifier le type en short et compteur ++** : il s'arrête au max du short qui est 32767 et puis il passe à la première valeur de type int avec une incrémentation et dans ce cas la condition n'est pas vérifiée et la boucle s'arrête !
3. **Modifier la condition : >-99999 et compteur --** : la condition est toujours vérifiée et il ne va jamais s'arrêter !!!!!

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int compteur = 10;
    printf("Explosion dans ");
    for( compteur = 10 ; compteur>0 ; compteur --)
    {
        printf("\n%d", compteur);
    }
    printf("\n\n BOOM");
    return 0;
}
```

Exemple

Les boucles

Exercice : code PIN

- 1- Créer un programme qui demande à l'utilisateur de rentrer le code PIN à 4 chiffres. Si il trouve le bon code, afficher le message : téléphone déverrouillé sinon afficher un message d'erreur et redemander d'entrer le code.
- 2- Gérer un nombre maximum de tentatives: Exemple 3 tentatives

```
Code PIN: 1234
  Erreur, il vous reste 2 tentatives.
Code PIN: 0000
  Telephone deverrouille
```

- 1- On ne connaît pas le nombre d'itération mais on souhaite entrer dans la boucle au moins une fois. On utilise alors une boucle "Do While"
- 2- Créer une constante pour le nombre maximum de tentatives.

Les boucles

Solution : code PIN

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    const int CODE_PIN = 0000;
    const int MAX_TENTATIVE = 3;
    int saisie;
    int nbre_Tentative= MAX_
TENTATIVE;
    // lecture de la saisie

    do {
        printf("Code Pin : ");
        scanf("%d",&saisie);

        if(saisie==CODE_PIN)
        {
            printf("\tTelephone deverrouille\n");
        }
        else
        {
            nbre_Tentative --;
            printf("\tErreur , il vous reste %d tentatives \n", nbre_Tentative);
        }
    }
    while(saisie!=CODE_PIN && nbre_Tentative > 0 );
    return 0;
}
```

Code PIN: 1234
Erreur, il vous reste 2 tentatives.
Code PIN: 0000
Telephone deverrouille

Les boucles

Break et Continue

break : sortir de la boucle courante

<pre>int main() { int var ; printf("debut \n"); for (var =0 ; var<5 ; var++) { if(var ==2) { break; } printf("%d\n",var); } printf("fin \n"); return 0; }</pre>	<pre>debut 0 1 fin</pre>
--	--------------------------

break

continue : sauter a la fin de la boucle

<pre>int main() { int var ; printf("debut \n"); for (var =0 ; var<5 ; var++) { if(var ==2) { continue; } printf("%d\n",var); } printf("fin \n"); return 0; }</pre>	<pre>debut 0 1 3 4 fin</pre>
---	------------------------------

continue

Les boucles

Break et Continue : exemple

```
int main()
{
    const int CODE_PIN = 0000;
    const int MAX_TENTATIVE = 3;
    int saisie_utilisateur;
    int nbre_Tentative= 0;
    int compteur;
    for( nbre_Tentative= 1 ;nbre_Tentative<=MAX_TENTATIVE ; nbre_Tentative++)
    {
        printf("Code Pin : ");
        scanf("%d",&saisie_utilisateur);
        if(saisie_utilisateur<0)
        {
            printf("code utilisateur doit etre superieur a 0 \n");
            continue;
        }
        if(saisie_utilisateur==CODE_PIN)
        {
            printf("\t Bienvenue\n");
            break;
        }
        else
            printf("\t Code incorrect  \n");
    }
    return 0;
}
```

Les boucles

Les boucles imbriquées

Exercice : afficher toutes les tables de multiplication de 1 à 10

Aide : commencer par afficher une seule table de multiplication

Table de 1 1 x 1 = 1 1 x 2 = 2 1 x 3 = 3 1 x 4 = 4 1 x 5 = 5 1 x 6 = 6 1 x 7 = 7 1 x 8 = 8 1 x 9 = 9 1 x 10 = 10	Table de 2 2 x 1 = 2 2 x 2 = 4 2 x 3 = 6 2 x 4 = 8 2 x 5 = 10 2 x 6 = 12 2 x 7 = 14 2 x 8 = 16 2 x 9 = 18 2 x 10 = 20	Table de 3 3 x 1 = 3 3 x 2 = 6 3 x 3 = 9 3 x 4 = 12 3 x 5 = 15 3 x 6 = 18 3 x 7 = 21 3 x 8 = 24 3 x 9 = 27 3 x 10 = 30	Table de 4 4 x 1 = 4 4 x 2 = 8 4 x 3 = 12 4 x 4 = 16 4 x 5 = 20 4 x 6 = 24 4 x 7 = 28 4 x 8 = 32 4 x 9 = 36 4 x 10 = 40	Table de 5 5 x 1 = 5 5 x 2 = 10 5 x 3 = 15 5 x 4 = 20 5 x 5 = 25 5 x 6 = 30 5 x 7 = 35 5 x 8 = 40 5 x 9 = 45 5 x 10 = 50
Table de 6 6 x 1 = 6 6 x 2 = 12 6 x 3 = 18 6 x 4 = 24 6 x 5 = 30 6 x 6 = 36 6 x 7 = 42 6 x 8 = 48 6 x 9 = 54 6 x 10 = 60	Table de 7 7 x 1 = 7 7 x 2 = 14 7 x 3 = 21 7 x 4 = 28 7 x 5 = 35 7 x 6 = 42 7 x 7 = 49 7 x 8 = 56 7 x 9 = 63 7 x 10 = 70	Table de 8 8 x 1 = 8 8 x 2 = 16 8 x 3 = 24 8 x 4 = 32 8 x 5 = 40 8 x 6 = 48 8 x 7 = 56 8 x 8 = 64 8 x 9 = 72 8 x 10 = 80	Table de 9 9 x 1 = 9 9 x 2 = 18 9 x 3 = 27 9 x 4 = 36 9 x 5 = 45 9 x 6 = 54 9 x 7 = 63 9 x 8 = 72 9 x 9 = 81 9 x 10 = 90	Table de 10 10 x 1 = 10 10 x 2 = 20 10 x 3 = 30 10 x 4 = 40 10 x 5 = 50 10 x 6 = 60 10 x 7 = 70 10 x 8 = 80 10 x 9 = 90 10 x 10 = 100

Les boucles

Les boucles imbriquées

Exercice : afficher toutes les tables de multiplication de 1 à 10

Aide : commencer par afficher une seule table de multiplication

```
int main()
{
    int ligne = 0;
    int id_table = 1;

    for(id_table=1; id_table<=10; id_table++)
    {

        printf("table multiplication %d\n", id_table);
        printf("*****\n");
        for(ligne=1; ligne<=10; ligne++)
        {
            printf("\t%d x %d = %d\n", ligne, id_table, ligne*id_table);
        }
        printf("\n") ;
    }
    return 0;
}
```

Les boucles

Jeux kahoot

<https://create.kahoot.it/details/boucles/db371927-a176-4c1f-8811-2b3b73759cdb>

[Kahoot!](#)

Les boucles

TP : un jeu

Nous allons créer un petit jeu de type “Le juste prix”. Pour les plus jeunes, cela correspond à un ancien jeu télévisé qui consiste à demander, aux participants, de trouver le prix d'un objet. Ils font alors des propositions et le présentateur leur dit simplement "c'est plus" ou "c'est moins", jusqu'à ce que l'un d'entre eux trouve le bon prix. Mais biensure, il y a une limite de temps! Nous allons donc demander à l'utilisateur de trouver un nombre entier compris entre 1 et 100.

Il vas alors pouvoir faire des propositions et pour chacune d'entre elles, nous allons afficher “c’est plus !” ou “c’est moins !”. lorsque l'utilisateur a trouvé le bon nombre alors on affiche un message de félicitation et on termine le programme. Le joueur a le droit à 10 tentatives.

```
...  
> 23  
c'est plus !  
> 30  
c'est moin !  
> 25  
Bravo! le nombre mystère est bien 25
```

Les boucles

TP : un jeu

Instructions

1- Créer la boucle de jeu

1.1 Demander à l'utilisateur de rentrer un entier pour trouver le nombre mystère.

1.2 Comparer la valeur saisie avec le nombre à trouver.

1.3 Gérer la fin de partie (le joueur a trouvé le bon nombre).

2- Ajouter un compteur qui indique combien de tentative a fait le joueur.

...

tentative 3 > 23

c'est plus !

tentative 4 > 30

c'est moins !

tentative 5 > 25

Bravo! le nombre mystère est bien 25

Les boucles

TP : un jeu

Pour aller plus loin

A- Modifier le code pour rendre le jeu plus difficile. pour cela, ajouter un nombre maximum de tentative.

B- Ajouter un menu à l'utilisateur en fin de partie lui permettant de faire une nouvelle partie ou quitter.

```
...
tentative 3/10 > 23
c'est plus !
tentative 4/10 > 30
c'est moin !
tentative 5/10 > 25
Bravo! le nombre mystère est bien 25

Voulez-vous faire une nouvelle partie (1-oui, 2-non)
> 2
A bientôt !
```


Les boucles

TP : un jeu

De l'Aide

1. Faire une boucle avec pour condition le fait que la saisie utilisateur corresponde ou non au nombre à trouver.
2. Il faut créer une nouvelle variable initialisé a 0 et l'incrémenter à chaque tentative de l'utilisateur.

A- Créer une nouvelle constante qui comporte le nombre maximum de tentative. quand ce nombre est atteint, utiliser une des instruction suivante pour terminer la partie (break, continue).

B- Mettre la boucle de jeu dans une nouvelle boucle. la condition de cette nouvelle boucle doit vérifier le choix de l'utilisateur.